

作业13

姓名：王泽黎 学号：2022K8009929011

13.1

假设文件系统的块大小为 4 KB，写入数据大小为 12 KB，即 3 个块。首先，创建 fs03.pdf 需要写入 4 个块：i-node bitmap，文件 i-node，目录块，目录 i-node。写文件需要写回 3 个块：data-block bitmap，文件 i-node，数据块。因此，共需要写入 7 个块。如果在任意时刻发生宕机，可能有如下情况：

- 如果在创建文件时宕机，由于写回顺序可能是任意的，可能有如下情况：
 - FS 不一致
 - i-node分配相关：
 - i-node bitmap未更新但i-node已分配
 - i-node bitmap已更新但i-node未分配
 - 目录分配相关：
 - 目录块未分配但目录i-node已分配
 - 目录块已分配但目录i-node未分配
- 如果在写文件时宕机，由于写回顺序可能是任意的，可能有如下情况：
 - FS 不一致：
 - data-block bitmap 与文件 i-node 中有一个没写回
 - 数据与元数据不一致：
 - i-node未更新但数据块已分配
 - i-node已更新但数据块未分配

13.2

(1) 如果文件系统采用数据日志，则步骤为：

1. 写 Begin Trans. 日记标记
2. 将修改文件的 i-node 写入日志
3. 将文件块 0 的新值写入日志
4. 将文件块 1 的新值写入日志
5. 写 commit 日志标记
6. 将修改文件的 i-node 写入磁盘

7. 将文件块 0 的新值写入磁盘
8. 将文件块 1 的新值写入磁盘
9. 清除日志

宕机情况分析：

1. 在步骤 4 之前宕机，由于磁盘上没有 commit 日志标记，系统什么都不做，宕机恢复后，文件 A 仍然是旧值。
2. 在步骤 4 之后，步骤 7 之前宕机，宕机恢复会发现 commit 日志标记，系统会将文件块 0 和文件块 1 的新值写入磁盘，文件 A 会是新值。
3. 在步骤 7 之后宕机，由于宕机前文件 A 的新值已经写入磁盘，文件 A 会是新值。

(2) 如果文件系统采元数据日志并且采用先改数据再改元数据的方式，则步骤为：

1. 将文件块 0 和 1 的新值写入磁盘
2. 写 Begin Trans. 日记标记
3. 将修改文件的 i-node 写入日志
4. 写 commit 日志标记
5. 将修改文件的 i-node 写入磁盘
6. 清除日志

宕机情况分析：

1. 在步骤 1 之前宕机，由于数据块尚未写入磁盘，系统什么都不做，宕机恢复后，文件 A 仍然是旧值。
2. 在步骤 1 过程中宕机，由于写入内容不确定，文件 A 的块 0 和 1 可能是旧值，也可能是新值。
3. 在步骤 1 之后，步骤 4 之前宕机，宕机恢复未发现 commit 日志标记，文件 A 是新值，但元数据未更新。
4. 在步骤 4 之后，步骤 6 之前宕机，宕机恢复会发现 commit 日志标记，文件 A 的数据和元数据都是新值。
5. 在步骤 6 之后宕机，由于宕机前已完成所有操作，文件 A 的数据和元数据都是新值。

13.3

(1) $\text{imap 块数} = \lceil 2000000 \div (4\text{KB} \div 4\text{B}) \rceil = \lceil 1953.125 \rceil = 1954$ 块

(2) $\text{CR 块数} = \lceil 1954 \div (4\text{KB} \div 4\text{B}) \rceil = \lceil 1.909 \rceil = 2$ 块

(3) ①先读 CR，把 CR 缓存在内存中；②根据 $\text{ino} = 356302$ ，得知对应 inode 在第 347 imap 块；③查 CR 得到 imap 块的磁盘地址；④读取 imap 块，在块内偏移 750 处找到对应 inode 的磁盘地址

13.4

(1) 文件大小: $10\text{MB} = 10240\text{KB} = 2560\text{个}4\text{KB块}$, 则需要10个直接指针(0-9号块), 1个一级间接指针(10-1033号块), 1个二级间接指针(1034-2559号块)

(2) 首先需要读取 CR 和 ino, 然后根据 ino 得到 imap 块的磁盘地址, 读取 imap 块, 找到对应 inode 的磁盘地址, 读取 inode。然后舍弃原先的 inode 和数据块, 一次写入新的 inode 和数据块。共需要 5 次磁盘 I/O。

(3) 首先需要读目录 inode 和根据目录块读取文件 inode, 利用 inode 访问二级间址指针和一级间址指针, 找到文件 foo 的第 2560 块, 然后写入新的 inode 和数据块。共需要 6 次磁盘 I/O。

(4)

1. 读目录 inode 和根据目录块读取文件 inode (两次)
2. 利用 inode 访问二级间址指针和一级间址指针 (两次)
3. 写入新的数据块 (一次)
4. 写 Begin Trans. 日记标记 (一次)
5. 将修改文件的 i-node 写入日志 (一次)
6. 写 commit 日志标记 (一次)
7. 将修改文件的 i-node 写入磁盘 (一次)
8. 清除日志 (一次)

共需要 10 次磁盘 I/O